



ELIMMEKATRUCK

Camion Uber — Plateforme B2B de Logistique

CAHIER DES CHARGES

VERSION

1.0

DATE

14 Mars 2026

STATUT

Document Vivant

BRANCHE

feat/sprint4

CONFIDENTIEL — USAGE INTERNE

Table des Matières

1. Présentation Générale du Projet	3
1.1 Objectif du système	3
1.2 Problème résolu	3
2. Description des Utilisateurs	4
2.1 Types d'utilisateurs (rôles)	4
2.2 Permissions principales	4
3. Fonctionnalités Principales	5
Module 1 — Authentification	5
Module 2 — Gestion des Organisations	5
Module 3 — Gestion de la Flotte	6
Module 4 — Cycle de Vie des Expéditions	6
Module 5 — Suivi GPS Temps Réel	7
Module 6 — Notifications	7
4. Architecture Technique	8
4.1 Vue d'ensemble	8
4.2 Technologies utilisées	8
4.3 Styles architecturaux	9
5. Modélisation des Données	10
5.1 Entités principales	10
5.2 Relations entre entités	11
6. API et Communication Entre Services	12
7. Sécurité du Système	14
8. Contraintes Techniques	15
9. Diagramme Logique du Système	16
10. Cahier des Charges Final	17
10.1 Contexte et objectif	17
10.2 Exigences fonctionnelles	17
10.3 Exigences non-fonctionnelles	18



Présentation Générale du Projet

Contexte, objectif et valeur métier de la plateforme

1.1 Objectif du Système

Elimmekatruck est une plateforme numérique B2B de mise en relation logistique. Elle connecte des **entreprises expéditrices** — qui ont des marchandises à transporter — avec des **transporteurs** — qui disposent d'une flotte de camions et de chauffeurs — dans le but de faciliter la gestion, le suivi et l'exécution d'expéditions de fret en Afrique centrale et de l'Ouest.

Positionnement produit : Le concept s'inspire du modèle de mise en relation à la demande (type Uber) appliqué au transport de fret B2B. L'objectif est de réduire les délais de mise en relation, d'augmenter le taux d'utilisation des flottes et de digitaliser le processus de bout en bout.

1.2 Problème Résolu

Avant cette plateforme, une entreprise cherchant un transporteur disponible devait contacter plusieurs opérateurs par téléphone, sans visibilité sur :

- la disponibilité réelle des camions et leur capacité de charge,
- la position géographique des véhicules,
- le statut d'avancement d'une livraison en cours.

Les transporteurs, de leur côté, ne disposaient d'aucun outil centralisé pour gérer leur flotte, leurs chauffeurs, et recevoir des missions de manière structurée.

Elimmekatruck résout ce problème en automatisant l'ensemble du cycle logistique :



Publication d'annonces

- Saisie structurée des besoins de transport
- Informations cargo, géolocalisation, prix



Matching automatique

- Correspondance poids / capacité camion
- Filtrage par ville et pays de départ



Gestion de flotte

- Suivi des véhicules et chauffeurs
- Statuts en temps réel (disponible / occupé)



Suivi GPS temps réel

- Positions GPS via WebSocket
- Historique complet de chaque expédition



Notifications automatiques

- E-mail et SMS à chaque étape clé
- File de messages asynchrone (BullMQ)



Multi-tenant

- Isolation complète des données par organisation
- Gestion de profils légaux locaux (RCCM, IFU)

2.1 Types d'Utilisateurs (Rôles)

Le système définit quatre rôles portés par le JWT et propagés via le header `x-user-role` dans chaque requête inter-services.

RÔLE	DESCRIPTION	ACCÈS DONNÉES
COMPANY	Représentant d'une entreprise expéditrice. Crée et suit ses propres expéditions.	Ses expéditions uniquement (<code>companyId = userId</code>)
TRANSPORTER	Opérateur d'une société de transport. Gère sa flotte (camions, chauffeurs) et accepte des missions.	Sa flotte et expéditions acceptées (<code>tenantId</code>)
DRIVER	Chauffeur rattaché à un transporteur. Démarre/termine des livraisons, envoie des positions GPS.	Expéditions assignées (<code>driverId = userId</code>)
ADMIN	Administrateur système. Accès total à toutes les ressources de tous les tenants.	Toutes les données, tous les tenants

2.2 Permissions Principales

ACTION	COMPANY	TRANSPORTER	DRIVER	ADMIN
Créer une expédition	✓	✗	✗	✓
Annuler une expédition (les siennes)	✓	✗	✗	✓
Rechercher des camions disponibles	✓	✓	✗	✓
Accepter une expédition	✗	✓	✗	✓
Démarrer / terminer une livraison	✗	✗	✓	✓
Envoyer une position GPS	✗	✗	✓	✓
Gérer camions et chauffeurs	✗	✓ (les siens)	✗	✓
Consulter l'historique de suivi	✓	✓	✓	✓
Modifier un profil d'organisation	✓ (le sien)	✓ (le sien)	✗	✓

Isolation multi-tenant : Chaque utilisateur ne voit que les données de son propre tenant, identifié par le champ `tenantId` présent dans le JWT et propagé en header `x-tenant-id`. Les administrateurs constituent la seule exception.

Module 1 — Authentification et Gestion des Identités (auth-service)

FONCTIONNALITÉ	DESCRIPTION
Inscription	Création d'un compte <code>COMPANY</code> ou <code>TRANSPORTER</code> — validation Zod, hachage bcrypt (12 rounds)
Connexion	Émission d'un JWT (RS256, 1 h) + refresh token, vérification du mot de passe haché
Déconnexion	Révocation immédiate du token via liste noire Redis (hash SHA-256, TTL automatique)
Renouvellement de token	Rotation du refresh token : l'ancien est révoqué, un nouveau JWT + refresh token est émis
Réinitialisation de mot de passe	Envoi d'un lien par e-mail contenant un token temporaire, confirmation avec nouveau mot de passe
Création compte chauffeur	Création d'un compte <code>DRIVER</code> rattaché à un transporteur (même <code>tenantId</code>)

Module 2 — Gestion des Organisations (company-service)

FONCTIONNALITÉ	DESCRIPTION
Profil entreprise expéditrice	CRUD complet : raison sociale, RCCM, IFU, secteur d'activité, représentant légal, coordonnées, logo
Profil transporteur	CRUD complet — même structure, type <code>TRANSPORTER</code>
Suspension	Soft-delete via <code>statut → SUSPENDED</code> — historique conservé
Consultation par tenant	Lookup par <code>tenantId</code> pour les appels inter-services (shipment récupère l'e-mail de contact)
Pagination et tri	Query params <code>page</code> , <code>limit</code> , <code>sortBy</code> , <code>sortOrder</code> sur les listes

Module 3 — Gestion de la Flotte (fleet-service)

FONCTIONNALITÉ	DESCRIPTION
Gestion des camions	CRUD : immatriculation, châssis, marque, modèle, type, gabarit, capacité max (kg), ville et pays de base
Gestion des statuts	Transition automatique : <code>AVAILABLE ↔ BUSY ↔ MAINTENANCE</code> lors des affectations
Gestion des chauffeurs	CRUD : nom, prénom, e-mail, téléphone, numéro de permis, statut
Affectation chauffeur ↔ camion	Associe un <code>driverId</code> à un camion (un camion = un chauffeur actif à la fois)
Algorithme de matching	Recherche des camions <code>AVAILABLE</code> filtrés par poids requis ≤ capacité, ville et pays de départ

Module 4 — Cycle de Vie des Expéditions (shipment-service)

FONCTIONNALITÉ	DESCRIPTION
Création d'une annonce	Saisie : marchandise, emballage, quantité, poids, villes départ/arrivée, date/heure, géolocalisation optionnelle, prix
Recherche de camions correspondants	Appel au fleet-service avec circuit breaker pour trouver les camions capables de prendre la charge
Acceptation d'une expédition	Le transporteur sélectionne l'annonce → statut ACCEPTED , e-mail envoyé aux deux parties
Démarrage de la livraison	Le chauffeur confirme le départ → statut IN_PROGRESS
Fin de livraison	Le chauffeur confirme l'arrivée → statut DELIVERED
Annulation	L'entreprise ou l'admin annule → statut CANCELLED avec commentaire obligatoire
Filtrage par rôle	COMPANY : ses expéditions. TRANSPORTER : expéditions acceptées. DRIVER : expéditions assignées.

CYCLE DE VIE DES STATUTS



Module 5 — Suivi GPS Temps Réel (tracking-service)

FONCTIONNALITÉ	DESCRIPTION
Enregistrement d'une position	POST <code>{ truckId, shipmentId, latitude, longitude, vitesse }</code> par le chauffeur
Historique d'un camion	Toutes les positions d'un camion, ordonnées par timestamp décroissant
Historique d'une expédition	Toutes les positions associées à un <code>shipmentId</code>
Dernière position connue	Dernier point GPS d'un camion (quasi temps réel)
Streaming WebSocket	Abonnement à un <code>shipmentId</code> ou <code>truckId</code> → push des nouvelles positions à tous les abonnés
Heartbeat WebSocket	Ping serveur toutes les 30 s — clients non-répondants déconnectés automatiquement

Module 6 — Notifications (notification-service)

FONCTIONNALITÉ	CANAL	TRAITEMENT
Envoi d'e-mail	SMTP Gmail via Nodemailer	BullMQ asynchrone
Envoi de SMS	Twilio API	BullMQ asynchrone
Notification push	À définir (stub présent)	Asynchrone
Retry automatique	Tous canaux	3 tentatives, backoff exponentiel 2s→4s→8s
Historique	Tous canaux	Stockage MongoDB, query par <code>userId</code>
Templates	E-mail / SMS	Création et stockage de modèles réutilisables

4.1 Vue d'Ensemble

Le système est une **architecture microservices** composée de **7 services indépendants** en Node.js/TypeScript, exposés derrière un **API Gateway unique** (deux instances en production pour la haute disponibilité). Les services communiquent via HTTP interne (réseau Docker). Les mises à jour temps réel utilisent WebSocket.

SERVICE	PORT HTTP	PORT WS	RÔLE	LIMITE MÉMOIRE
api-gateway ×2	3006	—	JWT, rate limiting, reverse proxy	256 Mo × 2
auth-service	3000	—	Identités, JWT, mots de passe	384 Mo
company-service	3001	—	Profils organisations	256 Mo
shipment-service	3002	—	Cycle de vie des expéditions	384 Mo
fleet-service	3003	—	Flotte camions et chauffeurs	256 Mo
tracking-service	3004	3007	GPS et temps réel	384 Mo
notification-service	3005	—	E-mail, SMS, BullMQ	384 Mo
MongoDB 7	27017	—	Replica set rs0, 6 bases logiques	512 Mo
Redis 7	6379	—	Rate limiting + BullMQ backend	128 Mo

4.2 Technologies Utilisées



Runtime & Langage

- Node.js 20 LTS
- TypeScript 5 (strict mode)
- Express 5
- Zod (validation des entrées)



Persistence

- MongoDB 7 — Mongoose 9
- Replica set rs0 (3 nœuds)
- Redis 7 (cache + queue)
- migrate-mongo (migrations)



Sécurité

- JWT RS256 — jsonwebtoken
- bcrypt (12 rounds)
- Helmet, CORS, rate-limit-redis
- TLS 1.2/1.3 via Nginx



Temps Réel & Queue

- WebSocket — bibliothèque ws
- BullMQ 5 (file de messages)
- Nodemailer (SMTP Gmail)
- Twilio SDK (SMS)



Observabilité

- Pino (logs JSON structurés)
- OpenTelemetry → Jaeger (traces)
- prom-client → Prometheus



Déploiement

- Docker (Alpine node:20-alpine)
- docker-compose + .prod.yml
- Nginx (TLS, HSTS, OCSP)

- Loki + Promtail + Grafana

- GitHub Actions (CI/CD)

4.3 Deux Styles Architecturaux Coexistants



Clean Architecture

auth-service, notification-service

```
domain/
├── entities/
├── value-objects/
├── repositories/ (interfaces)
└── errors/
application/
└── use-cases/
infrastructure/
├── db/ (Mongoose)
├── http/ (Express)
└── services/ (Nodemailer, Twilio)
```



Feature-Based Layered

company, fleet, shipment, tracking

```
src/<entity>/
├── <entity>.model.ts
├── <entity>.repository.ts
├── <entity>.service.ts
├── <entity>.controller.ts
└── <entity>.router.ts
```

Package partagé : Les services feature-based utilisent `@jlb226/generic-service` (npm) qui fournit les interfaces `IRepository<T>` et `GenericService<T>`, réduisant le code répétitif dans chaque service.

5.1 Entités Principales

USER — AUTH-SERVICE

CHAMP	TYPE	CONTRAINTES
id	UUID	unique, requis
email	String	unique, indexé, requis
passwordHash	String	bcrypt 12 rounds
role	Enum	COMPANY TRANSPORTER DRIVER
tenantId	String	indexé — identifie l'organisation

ORGANIZATION — COMPANY-SERVICE

CHAMP	TYPE	CONTRAINTES
id	UUID	unique
tenantId	String	unique — lien vers User
type	Enum	COMPANY TRANSPORTER
statut	Enum	ACTIVE SUSPENDED
raisonSociale, formeJuridique	String	requis
rccm	String	requis — registre du commerce
ifu	String	requis — identifiant fiscal unique
pays, ville	String	requis
email, telephone	String	requis — contact principal
nomRepresentant, fonctionRepresentant	String	requis — représentant légal
logo	String (URL)	optionnel

TRUCK — FLEET-SERVICE

CHAMP	TYPE	CONTRAINTES
id	UUID	unique
tenantId	String	indexé — propriétaire transporteur
immatriculation, chassis	String	unique chacun

CHAMP	TYPE	CONTRAINTES
<code>capaciteMax</code>	Number	requis — en kg, critère de matching
<code>statut</code>	Enum	<code>AVAILABLE</code> <code>BUSY</code> <code>MAINTENANCE</code> , indexé
<code>driverId</code>	String	optionnel — chauffeur affecté
<code>villeBase</code> , <code>paysBase</code>	String	requis — indexés pour matching géographique

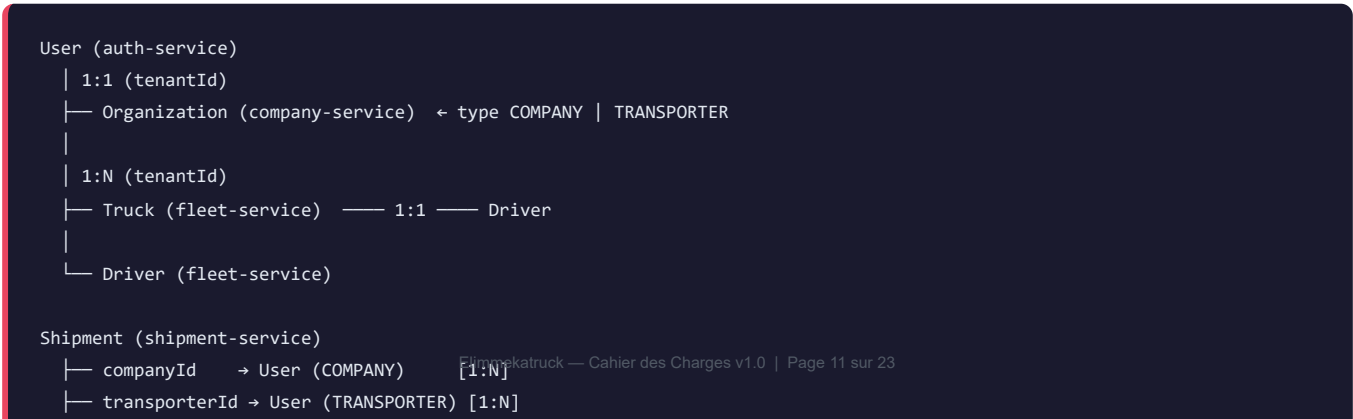
SHIPMENT — SHIPMENT-SERVICE

CHAMP	TYPE	CONTRAINTES
<code>id</code>	UUID	unique
<code>companyId</code>	String	indexé — User émetteur
<code>transporterId</code> , <code>truckId</code> , <code>driverId</code>	String	indexés, optionnels jusqu'à acceptation
<code>poids</code>	Number	requis — critère principal de matching
<code>villeDepart</code> , <code>paysDepart</code>	String	requis, indexés
<code>villeArrivee</code> , <code>paysArrivee</code>	String	requis
<code>geolocDepart</code> , <code>geolocArrivee</code>	<code>{lat, lng}</code>	optionnel — coordonnées GPS
<code>statut</code>	Enum	<code>PENDING</code> <code>ACCEPTED</code> <code>IN_PROGRESS</code> <code>DELIVERED</code> <code>CANCELLED</code>
<code>dateAnnonce</code> , <code>heureAnnonce</code>	Date / String	requis — format HH:MM

TRACKINGPOINT — TRACKING-SERVICE

CHAMP	TYPE	CONTRAINTES
<code>id</code>	UUID	unique
<code>truckId</code>	String	indexé — <code>{ truckId, timestamp desc }</code>
<code>shipmentId</code>	String	indexé — <code>{ shipmentId, timestamp desc }</code>
<code>latitude</code> , <code>longitude</code>	Number	requis, bornés (± 90 / ± 180)
<code>vitesse</code>	Number	optionnel, km/h, ≥ 0
<code>timestamp</code>	Date	requis

5.2 Relations entre Entités



```
|— truckId      → Truck      [N:1]
|— driverId     → Driver      [N:1]
└─ 1:N → TrackingPoint (tracking-service)
```

Notification (notification-service)

```
└─ recipientId → User      [N:1]
```

6.1 Flux de Requête Global

```
Client HTTP/WS
|
v
[Ngixn] - terminaison TLS, rate limiting (10/100 req/min), load balancing least_conn
|
v
[API Gateway]
| 1. Vérifie JWT (RS256 + clé publique RSA)
| 2. Vérifie liste noire Redis (hash SHA-256 du token)
| 3. Injecte headers : x-user-id, x-user-role, x-tenant-id
| 4. Proxy vers le service cible
v
[Service cible] - lit l'identité depuis les headers, applique filtres métier
```

6.2 Endpoints par Service

AUTH-SERVICE — /AUTH/*

MÉTHODE	CHEMIN	AUTH	DESCRIPTION
POST	/auth/signup	Non	Inscription (COMPANY ou TRANSPORTER)
POST	/auth/login	Non	Connexion → JWT + refresh token
POST	/auth/driver/create	Non	Création compte DRIVER
POST	/auth/logout	Oui	Révocation du token courant
POST	/auth/reset-password	Non	Demande de réinitialisation
POST	/auth/confirm-reset-password	Non	Confirmation avec token temporaire
POST	/auth/refresh	Non	Rotation refresh token → nouveau JWT

COMPANY-SERVICE — /COMPANY/*, /TRANSPORTER/*

MÉTHODE	CHEMIN	DESCRIPTION
GET	/company	Liste paginée des entreprises
POST	/company	Créer un profil entreprise
GET	/company/:id	Détail par UUID
PUT	/company/:id	Mise à jour
DELETE	/company/:id	Suspension (soft delete)

MÉTHODE	CHEMIN	DESCRIPTION
GET	/company/tenant/:tenantId	Lookup par tenantId (inter-services)

Même structure pour /transporter/*

FLEET-SERVICE — /TRUCKS/*, /DRIVERS/*

MÉTHODE	CHEMIN	RÔLES	DESCRIPTION
GET	/trucks/match? poids=&villeDepart=&paysDepart=	Tous	Algorithme de matching
POST	/trucks/assign-driver	TRANSPORTER, ADMIN	Affecter un chauffeur à un camion
GET POST	/trucks	Tous / TRANSPORTER	Liste / Création
GET PUT DEL	/trucks/:id	Rôle contrôlé	Détail / Mise à jour / Suppression

Même structure pour /drivers/*

SHIPMENT-SERVICE — /SHIPMENTS/*

MÉTHODE	CHEMIN	RÔLES	DESCRIPTION
GET	/shipments/search? poids=&villeDepart=&paysDepart=	Tous	Recherche de camions disponibles
POST	/shipments/:id/accept	TRANSPORTER, ADMIN	Accepter une expédition
POST	/shipments/:id/start	DRIVER, ADMIN	Démarrer la livraison
POST	/shipments/:id/deliver	DRIVER, ADMIN	Confirmer la livraison
GET POST	/shipments	COMPANY (POST), Tous (GET)	Liste / Création
DELETE	/shipments/:id	COMPANY, ADMIN	Annulation

TRACKING-SERVICE — /TRACKING/* + WEBSOCKET

PROTOCOLE	MÉTHODE	CHEMIN	DESCRIPTION
HTTP	POST	/tracking	Enregistrer une position GPS
HTTP	GET	/tracking/truck/:truckId	Dernière position connue
HTTP	GET	/tracking/truck/:truckId/history	Historique complet d'un camion
HTTP	GET	/tracking/shipment/:shipmentId	Positions d'une expédition
WS	—	ws://host:3007?token=<jwt>	Streaming temps réel par abonnement

DE	VERS	ENDPOINT APPELÉ	DÉCLENCHEUR
auth-service	notification-service	POST /notification/email	Reset mot de passe / bienvenue
shipment-service	fleet-service	GET /fleet/trucks/match	Recherche de camions — circuit breaker
shipment-service	company-service	GET /company/tenant/:id GET /transporter/tenant/:id	Récupération e-mail pour notification
shipment-service	notification-service	POST /notification/email	Changement de statut expédition
tracking-service	notification-service	POST /notification/email	Alertes GPS

Mécanismes de résilience sur les appels sortants :

- `fetchWithTimeout` — timeout 10 s sur tous les appels
- `fetchWithRetry` — 3 tentatives, backoff exponentiel avec jitter
- **Circuit Breaker** (shipment→fleet) — 5 échecs → état OPEN 30 s → HALF_OPEN

7.1 Authentification

MÉCANISME	DÉTAIL
Algorithme JWT	RS256 — clé asymétrique (privée dans auth-service uniquement, publique dans tous les autres)
Durée de vie	Access token : 1 h / Refresh token : durée configurable par variable d'environnement
Révocation	Liste noire Redis (hash SHA-256 du token, TTL = durée de vie résiduelle)
Hachage mot de passe	bcrypt, 12 rounds — valeur obfusquée, jamais retournée en API
Rotation refresh token	L'ancien token est révoqué à chaque renouvellement (rotation stricte)

7.2 Autorisation

MÉCANISME	DÉTAIL
Propagation d'identité	La gateway injecte <code>x-user-id</code> , <code>x-user-role</code> , <code>x-tenant-id</code> sur chaque requête proxyée
Contrôle d'accès	Chaque contrôleur vérifie le rôle avant d'exécuter l'action métier
Isolation tenant	Les requêtes de lecture/écriture sont filtrées par <code>tenantId</code> dans chaque service
Isolation WebSocket	Un DRIVER ne peut s'abonner qu'aux expéditions où <code>driverId === userId</code>

7.3 Défense en Profondeur

COUCHE	MÉCANISME	CONFIGURATION
Nginx	Rate limiting	Auth : 10 req/min API : 100 req/min WS : 10 connexions
API Gateway	Rate limiting Redis-backed	Fenêtre et maximum configurables par env
Auth Service	Rate limiting login	20 req / 15 min par IP
Tous services	Helmet	En-têtes sécurité HTTP, CSP (désactivée pour Swagger)
Tous services	CORS	<code>ALLOWED_ORIGINS</code> depuis env, credentials activés
Nginx (prod)	TLS	TLS 1.2+1.3, HSTS 63 072 000 s + preload, OCSP stapling
Nginx (prod)	En-têtes sécurité	HSTS, X-Content-Type-Options, X-Frame-Options, CSP: default-src 'self'
Docker	Conteneurs non-root	Utilisateur <code>appuser</code> , image Alpine node:20

8.1 Prérequis Système

COMPOSANT	VERSION MINIMALE	RÔLE
Node.js	20 LTS	Runtime des 7 services
MongoDB	7.0	Replica set <code>rs0</code> obligatoire pour les transactions Mongoose
Redis	7.0	Rate limiting partagé + backend BullMQ
Docker	24+	Conteneurisation
Docker Compose	2.x	Orchestration locale et production

8.2 Variables d'Environnement Critiques par Service

Chaque service valide ses variables au démarrage et lève une exception si une variable requise est absente (**fail-fast**).

SERVICE	VARIABLES CRITIQUES
api-gateway	<code>JWT_PUBLIC_KEY</code> , <code>REDIS_URL</code> , <code>ALLOWED_ORIGINS</code> , URLs des 6 services
auth-service	<code>JWT_PRIVATE_KEY</code> , <code>JWT_PUBLIC_KEY</code> , <code>JWT_REFRESH_SECRET</code> , <code>DATABASE_URL</code>
company-service	<code>MONGO_URI</code> , <code>JWT_PUBLIC_KEY</code>
fleet-service	<code>MONGODB_URI</code> , <code>JWT_PUBLIC_KEY</code>
shipment-service	<code>MONGODB_URI</code> , <code>JWT_PUBLIC_KEY</code> , <code>FLEET_SERVICE_URL</code> , <code>NOTIFICATION_SERVICE_URL</code> , <code>COMPANY_SERVICE_URL</code>
tracking-service	<code>MONGODB_URI</code> , <code>JWT_PUBLIC_KEY</code> , <code>WS_PORT</code> , <code>NOTIFICATION_SERVICE_URL</code>
notification-service	<code>DATABASE_URL</code> , <code>REDIS_URL</code> , <code>SMTP_HOST</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>TWILIO_ACCOUNT_SID</code> , <code>TWILIO_AUTH_TOKEN</code>

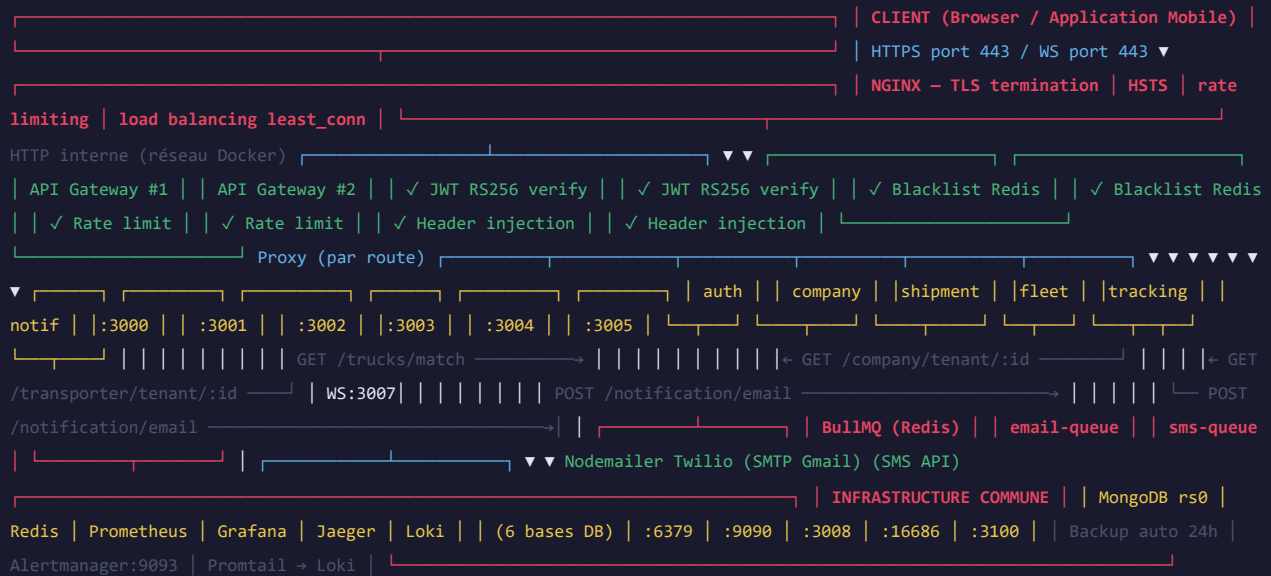
8.3 Pipeline CI/CD (GitHub Actions)

5 jobs exécutés en parallèle sur les 7 services à chaque push ou pull request :

JOB	COMMANDES	BLOQUANT
Tests + couverture	<code>npm test -- --forceExit --coverage</code>	Oui — seuil 60 % lignes/fonctions
Typage	<code>npx tsc --noEmit</code>	Oui
Lint	<code>npx eslint src --ext .ts</code>	Oui
Docker build	Build et cache des images	Oui
Audit sécurité	<code>npm audit --audit-level=high</code> + Trivy (CRITICAL/HIGH)	Oui

8.4 Observabilité

SIGNAL	OUTIL	ENDPOINT / PORT
Métriques	prom-client → Prometheus	<code>/metrics</code> sur chaque service, scrape toutes les 15 s
Logs	Pino → Promtail → Loki	JSON structuré, <code>x-request-id</code> dans chaque ligne
Traces	OpenTelemetry → Jaeger	Export OTLP HTTP vers <code>jaeger:4318</code>
Dashboards	Grafana	Port 3008 — sources Prometheus + Loki
Alerting	AlertManager	Port 9093 — notifications SMTP



10.1 Contexte et Objectif

La société **Elimmekatruck** développe et opère une plateforme numérique B2B de logistique, dénommée *Camion Uber*, permettant la mise en relation automatisée entre des entreprises expédiant des marchandises et des transporteurs disposant de flotte propre. La plateforme couvre l'intégralité du cycle logistique : de la publication d'une annonce de transport jusqu'à la livraison confirmée, avec suivi GPS en temps réel.

10.2 Exigences Fonctionnelles

EF-01 — Gestion des comptes

- Le système doit permettre l'inscription d'entreprises (**COMPANY**) et de transporteurs (**TRANSPORTER**).
- Un transporteur doit pouvoir créer des comptes chauffeurs (**DRIVER**) rattachés à son organisation.
- Le système doit proposer une procédure de réinitialisation de mot de passe par e-mail.
- Le système doit prendre en charge la déconnexion explicite (révocation immédiate du token).

EF-02 — Profils d'organisation

- Chaque utilisateur doit disposer d'un profil complet incluant les informations légales locales (RCCM, IFU), les coordonnées et les informations du représentant légal.
- Les organisations peuvent être suspendues — l'historique est conservé (soft-delete).

EF-03 — Gestion de flotte

- Un transporteur doit pouvoir enregistrer ses camions avec leur capacité de charge (kg), type, gabarit, et ville de base.
- Un camion doit pouvoir être associé à un chauffeur unique à la fois.
- Le statut des camions (**AVAILABLE** , **BUSY** , **MAINTENANCE**) doit être maintenu automatiquement.
- Un algorithme de matching doit retrouver les camions disponibles correspondant à un poids requis et une ville de départ.

EF-04 — Cycle de vie des expéditions

- Une entreprise doit pouvoir publier une annonce de transport (marchandise, poids, origine, destination, date).
- Le système doit proposer automatiquement les camions correspondants à l'annonce.
- Un transporteur doit pouvoir accepter une annonce, déclenchant son affectation à un camion et un chauffeur.
- Le chauffeur doit pouvoir démarrer et terminer la livraison depuis son interface.
- L'annulation doit être possible avec enregistrement d'un motif.
- Le système doit envoyer une notification (e-mail) aux parties concernées à chaque changement d'état.

EF-05 — Suivi GPS temps réel

- Le chauffeur doit pouvoir envoyer sa position GPS (latitude, longitude, vitesse) depuis son application.
- Toute partie prenante d'une expédition doit pouvoir consulter l'historique complet des positions.
- Les clients doivent recevoir les mises à jour de position en temps réel via WebSocket.

- Le système doit notifier par e-mail les parties concernées à chaque changement d'état d'une expédition.
- Le système doit supporter l'envoi de SMS via Twilio.
- Les notifications doivent être traitées de manière asynchrone avec 3 tentatives automatiques en cas d'échec.

10.3 Exigences Non-Fonctionnelles

ENF-01 — Performance

- L'API Gateway doit supporter au moins 100 requêtes simultanées sans dégradation notable.
- Les requêtes de matching de camions doivent répondre en moins de 500 ms sous charge normale.
- Le délai de diffusion WebSocket (position GPS → client) doit être inférieur à 2 s.

ENF-02 — Disponibilité

- Le système cible une disponibilité de 99,5 % en production.
- Chaque service doit implémenter un arrêt gracieux (drain des requêtes en cours avant extinction).
- Les health checks (`/health`) doivent retourner l'état de la base de données et du service.
- Le déploiement en production doit s'effectuer sans interruption de service (rolling update ou blue-green).

ENF-03 — Sécurité

- Toutes les communications externes doivent être chiffrées (TLS 1.2 minimum).
- Les tokens JWT doivent utiliser l'algorithme RS256 avec rotation des clés possible sans interruption.
- Le mot de passe doit être haché avec bcrypt (coût minimum : 12).
- Aucune donnée sensible (mot de passe, token) ne doit apparaître dans les logs.
- Les secrets de production ne doivent en aucun cas être commités dans le dépôt de code source.

ENF-04 — Isolation Multi-Tenant

- Chaque utilisateur ne doit accéder qu'aux données de son propre tenant.
- L'isolation doit être appliquée au niveau des requêtes base de données, pas uniquement au niveau des contrôleurs.
- Un test d'intégration doit valider qu'un tenant A ne peut pas lire les données du tenant B.

ENF-05 — Observabilité

- Chaque service doit exposer des métriques Prometheus sur `/metrics` .
- Les logs doivent être structurés en JSON et inclure un `requestId` corrélant toutes les entrées d'une même requête.
- Les traces distribuées doivent être exportées vers Jaeger via OpenTelemetry.
- Des alertes doivent être configurées : latence P99 > 1 s, taux d'erreur > 1 %, redémarrage de pods, spike d'erreurs > 10/min.

ENF-06 — Maintenabilité

- Chaque service doit avoir une couverture de tests automatisés d'au moins 75 %.
- Le pipeline CI/CD doit bloquer tout merge en cas d'échec des tests, du typage, du lint ou d'une vulnérabilité de sécurité haute/critique.
- Les migrations de base de données doivent être versionnées et rejouables.

ENF-07 — Déploiement

- Le système doit être entièrement conteneurisé et déployable via `docker-compose` .
- Les images Docker doivent s'exécuter avec un utilisateur non-root.
- Les ressources CPU et mémoire doivent être bornées pour chaque conteneur.

10.4 Contraintes de Conformité

- Les données de l'organisation (RCCM, IFU) sont des identifiants légaux propres aux pays d'Afrique centrale/occidentale et doivent être traités comme des données sensibles.
- Les données personnelles (e-mail, téléphone, numéro de permis) sont soumises aux réglementations locales de protection des données et ne doivent pas être exposées dans les logs applicatifs.
- Les secrets de production (clés JWT, credentials SMTP, tokens Twilio) ne doivent en aucun cas être commités dans le dépôt de code source — un gestionnaire de secrets externe est requis.